Université Abou Bakr Belkaid
Faculté des Sciences
Département d'informatique

Recherche Opérationnelle

Chap 2: Programmation Linéaire en nombres entiers et nombres booléennes

4éme Année Ingénieur (IA)*(IA/GL)*

2026-2027

Introduction

- La programmation linéaire en variables entières et en variables booléennes est essentielle, car la plupart des problèmes réels incluent des variables qui doivent, par nature, prendre une valeur entière ou bien booléenne.

- Variables entières : Nombre de villes, nombres d'étudiants….

- Variables booléennes: Etudiant présent/absent, Existe/n'existe pas…..

Introduction

- Si l'on tente d'arrondir ces valeurs à l'entier le plus proche, la solution obtenue peut s'éloigner de l'optimum réel ou, pire, devenir irréalisable.

- Dans ces cas, il est préférable d'utiliser une méthode plus avancée que le simplexe pour garantir des solutions valides et optimales.

Programmation Linéaire en nombres entiers

- Exemple:

$$
\begin{cases}
\text{Max } Z = 10x + 11y \\
10x + 12y \leq 59 \\
x \leq 2.5 \\
x, y \in N
\end{cases}
$$

-

Méthode Branch and Bound

Stratégie :

1. **Initialisation** : Commencer avec le problème global et, si possible, calculer une solution initiale pour établir une borne supérieure ou inférieure.
2. **Division** : Diviser le problème courant en plusieurs sous-problèmes.
3. **Calcul des Bornes** (maximisation) : La borne inférieure correspond à la meilleure solution réalisable trouvée jusqu'à présent, tandis que la borne supérieure est une estimation de la valeur optimale possible dans chaque sous-problème.
4. **Élagage** : Comparer les bornes avec la meilleure solution actuelle et éliminer les sous-problèmes inutiles.
5. **Sélection** : Choisir le prochain sous-problème à explorer, souvent selon des stratégies comme la recherche en profondeur ou en largeur.
6. **Répétition** : Répéter les étapes jusqu'à ce que tous les sous-problèmes soient résolus ou élagués.

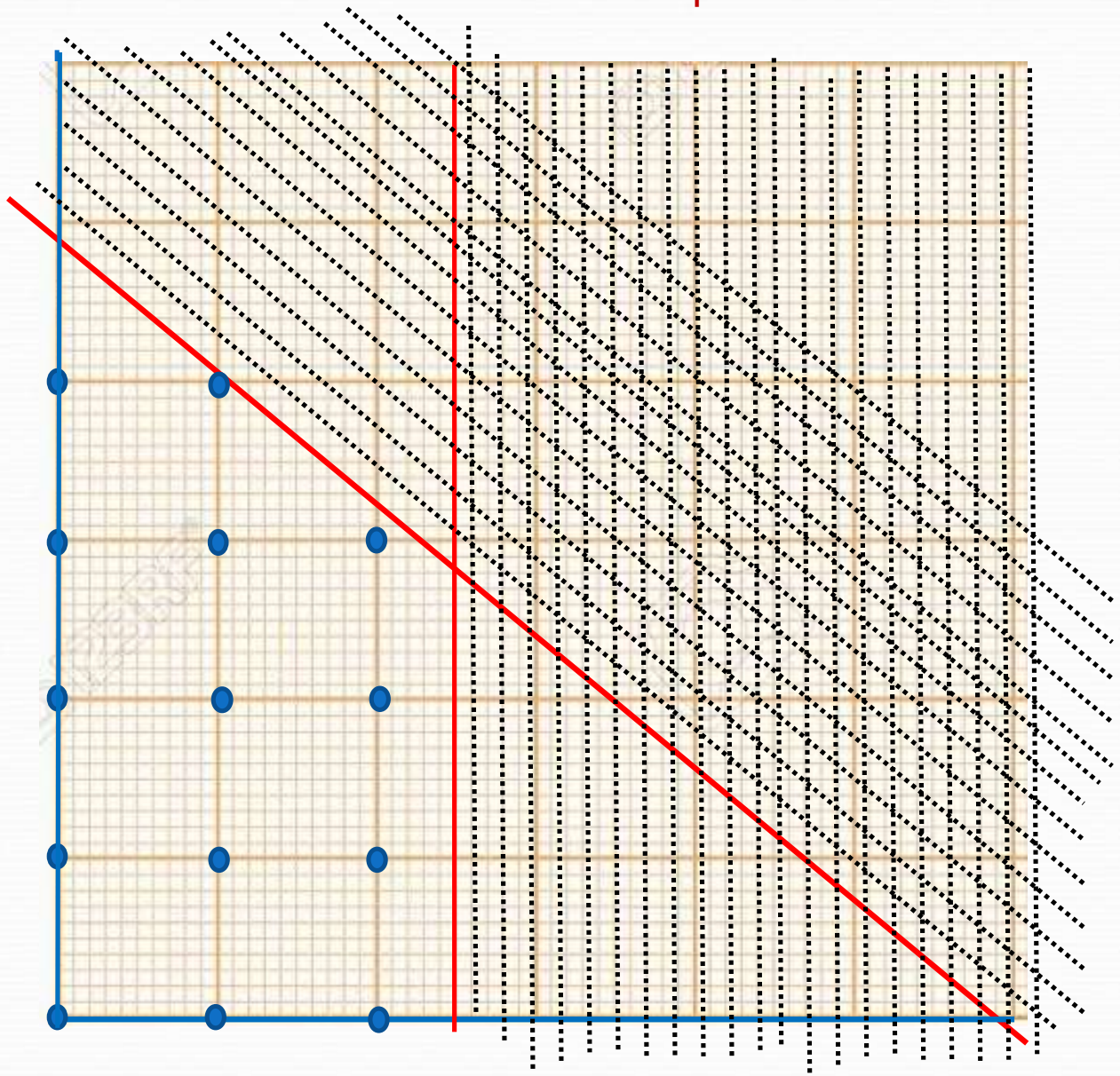
Méthode Branch and Bound

- **Avantages**
  - **Optimalité garantie** : L'algorithme trouve toujours la solution optimale si bien exécuté.
  - **Réduction de l'espace de recherche** : Exploite les caractéristiques du problème pour éliminer les branches inutiles.
- **Inconvénients**
  - Peut être coûteux en temps de calcul pour des problèmes de grande taille.
  - L'efficacité de l'algorithme **Branch and Bound** repose sur deux aspects essentiels : **comment le problème est divisé** et **comment calculer les bornes**.

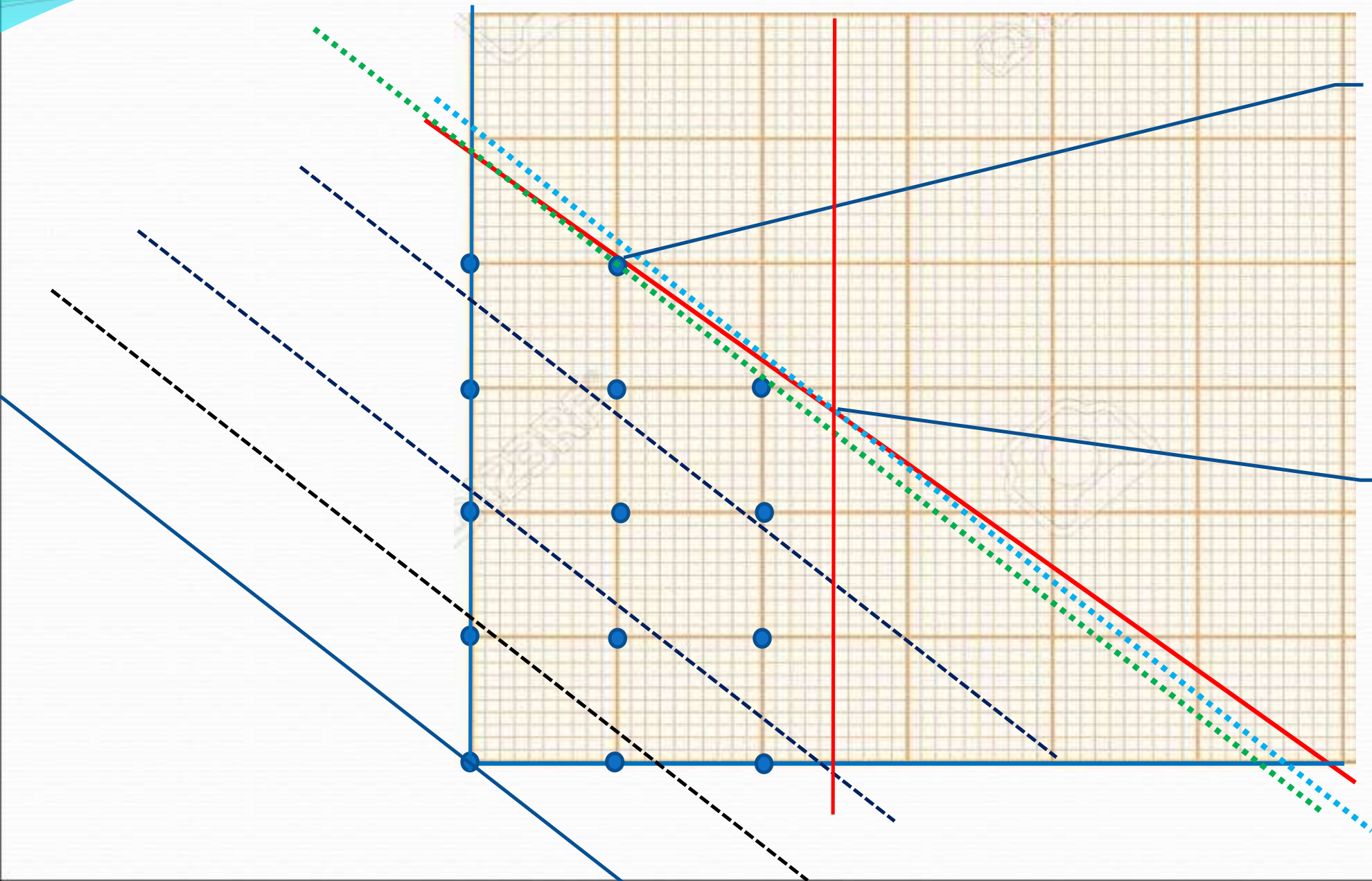
Pseudo-Code de la méthode Branch and Bound

1. **Entrée** : Problème d'optimisation (minimisation ou maximisation)
2. **Initialiser** la meilleure solution ← ∞ (si minimisation) ou -∞ (si maximisation)
3. Créer une **file** (ou pile, ou priorité) de nœuds à explorer
4. **Ajouter** le nœud racine (problème complet)
5. **Tant que** la file n'est pas vide :
6. Extraire un nœud courant de la file
7. **Si** le nœud courant représente une solution réalisable :
8. **Si** valeur_solution(nœud) est meilleure que meilleure solution :
9. meilleure solution ← valeur_solution(nœud)
10. **Sinon :**
11. Calculer une borne (inférieure ou supérieure selon le problème)
12. **Si** borne(nœud) est pire que meilleure solution alors Ignorer ce nœud (Bounding)
13. **Sinon :**
14. Diviser le problème courant en sous-problèmes (Branching)
15. Pour chaque sous-problème Ajouter le nouveau nœud à la file
16. **Fin Tant que**
17. **Retourner** meilleure solution

Programmation Linéaire en nombres entiers : Exemple



Programmation Linéaire en nombres entiers : Exemple



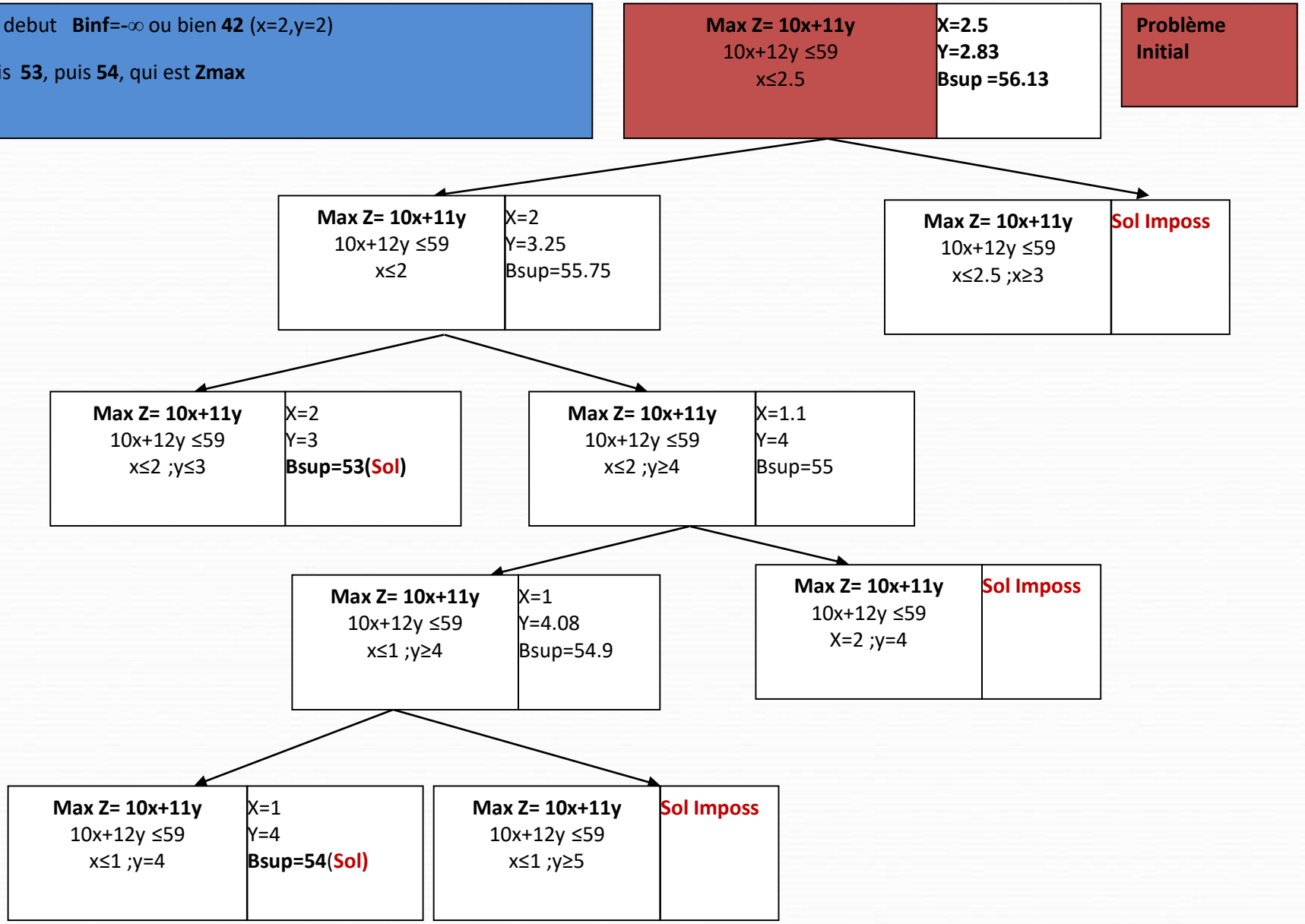
La solution du problème discret (x=1,y=4, Z=54).

La solution du problème continu (x=2.5,y=2.83, Z=56.13).

Solution

Au debut **Binf**=-∞ ou bien **42** (x=2,y=2)

Puis **53**, puis **54**, qui est **Zmax**



Programmation Linéaire en nombres Booléens:

Exemple: Sac à Dos

Objets	1	2	3	4
Poids	2	5	4	3
Bénéfices	5	8	7	6

$$\begin{cases} \text{Max } Z = 5x_1+8x_2+7x_3+6x_4 \\ 2X_1+5x_2+4x_3+3x_4 \leq 10 \\ X_i \in \{0,1\},\ i=1..4 \end{cases}$$

Programmation Linéaire en nombres Booléens:

Exemple: Sac à Dos

Exemple

Objets	1	2	3	4
Poids	2	5	4	3
Bénéfices	5	8	7	6
Benif/poids	2.5	1.6	1.8	2

- J'ai un sac à dos de capacité maximale k (poids). J'ai *n* objets de poids et de valeurs diverses.
- Je souhaite remplir le sac de façon à cumuler un maximum de valeurs.
- Quel est le bénéfice maximum que je peux avoir?
- **Dans cet exemple la capacité du sac est 10, et nous cherchons le B_max?**

Résolution avec les heuristiques:

- **1)Methode minimisant le poids→ 5+6+7=18**
- **2)Methode maximisant le benifices→ 8+7=15**
- **3) Maximisation Benifices/poids →5+7+6=18**

Sac à dos -programmation dynamique

Objets	1	2	3	4
Poids	2	5	4	3
Bénéfices	5	8	7	6

	0	1	2	3	4	5	6	7	8	9	10
Objet 1	0	0	5	5	5	5	5	5	5	5	5
Objet 2	0	0	5	5	5	8	8	13	13	13	13
Objet 3	0	0	5	5	7	8	8	13	13	15	15
Objet 4	0	0	5	6	7	11	11	13	14	15	19



- Résultats: objets(4,2,1)→ bénifice=6+8+5=19

Sac à dos -programmation dynamique

On note B(k, p) le bénéfice maximal réalisable avec des objets 1, 2, . . . , k et le poids maximal p

pour k = 1:

$$B(1, p) = \begin{cases} 0 \text{ si } p < p_1 \\ B(1, p) \text{ si } p > p_1 \end{cases}$$

Pour k > 1

$$B(k, p) = \begin{cases} B(k-1, p) & \text{si } p < p_k \\ \text{Max}(B(k-1, p), B(k-1, p - \text{poidsObjet}[k]) + \text{valeurObjet}[k]) & \text{sinon} \end{cases}$$

Complexité → **Θ(k*p)**

Programmation Linéaire en nombres Booléens:

Exemple: Sac à Dos(Branch and Bound)

